USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

Containment, and Why It Is Not Security

UM-NATOS-032

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-032	1.0 · 2026-08-18	**Shipped; verified on hardware**	Internal

1. Abstract

UM-NATOS-031 finished the device model: seven devices behind one syscall, an argument harness in front of them, and applications that can reach a light sensor, a bus, a keypad and a card. It closed the gap chapter 31 named. It also opened one, and named it in its own §4:

> Every application can reach every device.

That is now false. Each program declares which devices it may touch, the declaration lives next to its arena size in the launch table, and the check sits on all four routes from a program to hardware. A program with no declaration gets no hardware.

This report is short, because the feature is small — about seventy lines. Most of it is spent on two things worth more than the code: **why this must not be called security**, and the two ways it could have been quietly wrong.

2. What it does

A bitmap per caller. Bit *N* grants device *N*.

```
static uint32_t g_perms[DEVICE_CALLER_KERNEL + 1u];

static int permitted(uint32_t caller, uint32_t id)
{
    if (caller > DEVICE_CALLER_KERNEL || id >= 32u) { g_denials++; return 0; }
    if (!((g_perms[caller] >> id) & 1u))            { g_denials++; return 0; }
    return 1;
}
```

Checked at the top of `device_read`, `device_write`, `device_xfer_out` and `device_xfer_in` — which is every route from a program to a peripheral. Nothing else needs to change, and no driver knows this exists.

It is cheap because the hard part was built by accident. `device_t` grew a `caller` argument in UM-NATOS-031 so the `store` device could bank persistent slots per application. That means **every device call already carries who is asking** — and the caller comes from `vm->app_id`, the kernel's own record, never from a register the program controls. A program naming its own identity would be the same mistake as trusting an offset it supplied, and §2 of UM-NATOS-031 exists because of that mistake.

Grants come from the program table:

```
{ "dev", vm_app_dev, VM_APP_DEV_LEN, 768u, VM_APP_DEV_AT_PUBLISH,
  P_LIGHT | P_STORE | P_ECHO },
```

Next to the arena size, deliberately, so both limits on a program are visible in one place and reviewable without disassembling anything. Ten of the eleven entries are `DEV_PERM_NONE`, which is the point: a program nobody thought about gets nothing rather than everything.

A refusal is **not a fault**. The program is not terminated. This follows the rule UM-NATOS-031 §1 set for a bad channel — asking for something you were not granted is legal, and a program that cannot discover its own limits without dying cannot discover them.

---

3. What this is not

This is containment. It is not security, and no later report may call it security until one specific thing exists.

A permission grant is only meaningful if the image it applies to cannot be swapped for another. nat-os has no image identity: no signature, no content hash, nothing. Anyone able to flash the board can put any bytes behind any name in the program table, and those bytes inherit that name's grants.

What it does do is real and worth having:

- a buggy program cannot reach hardware it was never meant to touch
- the intended capability surface of every program is explicit and auditable
- the surface is *small* by default, so the interesting entries stand out

That is a containment property. It bounds accident, not intent. The distinction matters because this kernel has spent whole days on instruments that lied, and a feature described one notch stronger than it is becomes exactly that.

The same missing piece has a second consequence, recorded in `device.c` beside the `store` driver. A caller id is a **slot**, and slots are reused. A program granted `store` that lands in a slot an earlier program used will read what that program left behind. Permissions narrow it — the new tenant needs an explicit grant to read anything at all — but do not close it. It is not fixed by clearing the bank on retire, which would delete the persistence the device exists to provide. The bank wants to be keyed on *which program*, and nat-os cannot say which program. **Both holes are closed by image identity, and neither before it.**

---

4. The two ways this could have been quietly wrong

Neither of these is clever. Both are the kind of thing that works in every test you think to run and fails on a Tuesday.

4.1 A capability outliving its holder

`retire()` releases the arena and clears undelivered mail. It did not clear grants, because grants did not exist when it was written.

Application ids are slots and slots are reused. Without a revoke, a program granted the SD card would leave that grant sitting in the bank, and **the next program to land in the same slot would inherit hardware nobody granted it**. The symptom would appear in an unrelated program, on a run whose behaviour depended on what had exited earlier — which is close to the worst shape a defect can have.

```
for (int i = 0; i < APP_MAX; i++) {
    if (&g_apps[i] == a) {
        ipc_clear(i);
        device_grant((uint32_t)i, DEV_PERM_NONE);
    }
}
```

A capability that outlives its holder is not a capability.

4.2 Two launch paths, one grant

Three call sites reached `app_start()`: the shell's `run`, the shell's `shell_launch()` (which the desktop uses), and `start_program()` in `kmain.c` (which boots `ping` and `pong`).

A path that started a program without granting would produce one that silently could not reach hardware. A path that granted the *wrong table entry* would hand it someone else's capabilities. The second is worse, and it is precisely the failure mode UM-NATOS-017 already recorded once, when a hard-coded `PROGRAMS[4]` launched `gfxrogue` under the name `paint` after the table was reordered.

Both shell paths now go through one `launch_entry()` that starts and grants together; `start_program()` grants from the entry it matched by name. The boot path and the typed path must agree, or a program started at boot behaves differently from the same program started by typing its name.

---

5. Verification

Two scripts, `tools/serial/perms_test.py` and `tools/serial/perms_live.py`. Both drive one boot, because every claim here is about state that changes across commands and a fresh reset between steps would test none of it.

5.1 The grant is what the table says

```
$ run dev
   started id=0 perms=0x00000025
```

`0x25` is bits 0, 2 and 5 — `light`, `store`, `echo`. Not all seven. The program then enumerated the table, round-tripped a bulk transfer through `echo`, and took its sixteen light readings. It was never granted `beep`, which it used to seize, and never granted `sd` or `i2c`.

5.2 The check is live, not launch-time

This is the claim worth testing, and the first attempt to test it **failed to test anything** — a detail worth keeping. `app_dev` is 463 instructions end to end and finishes in well under a second. The script waited 1.2 s for it to "get going" and then revoked, by which point the program had already printed all sixteen readings

and exited. The revoke landed on an empty slot. Everything printed looked healthy, and it demonstrated nothing.

The fix is to send both lines in a single write, with no read between them: the shell consumes one line per poll, so the revoke lands a few instructions into the program rather than after it.

```
$ run dev
$ perms 0 0 off   (same write -- no gap)
   started id=0 perms=0x00000025
   revoked light for app 0
   [dev] device table: 7 entries
   [dev] 0 = light   ... 6 = sd
   [dev] bulk transfer round trip OK (arena -> echo -> arena)
                                     <-- and then nothing. Zero readings.
```

Enumeration still worked: `DEV_OP_COUNT`, `NAME` and `INFO` read the table, not the hardware, and are not permission-checked. The `echo` transfer still worked — that grant was untouched. The light readings stopped dead at the first refused read.

```
$ perms
   app  name        devices
   0    dev         store echo
   denials=48450
```

Alive, holding exactly the two devices it still had, and being refused continuously. Meanwhile the shell's own `dev` command read the light sensor fine — the shell passes `DEVICE_CALLER_KERNEL`, a different bank.

```
$ perms 0 0 on
   granted light for app 0
   [dev] light = 157
   ... sixteen readings ...
   [app 0 'dev' finished] status=0 after 477103 instructions
```

It resumed and completed. **477,103 instructions against a baseline of 463** — the spin is visible in the count, which is the cleanest evidence that the refusals were real and the program genuinely survived them.

5.3 A slot does not inherit

```
$ perms 0 6 on          <- grant the SD card the table never gave it
$ perms
   0    dev         store echo sd
$ kill 0
$ run counter
   started id=0 perms=none
$ perms
   0    counter     (none)
```

5.4 A denied program does not starve the system

`vm.c` decides whether to end the caller's slice from `device_is_slow(id)` — the device's own flag — **not** from whether the call succeeded. So a program being refused a slow device still yields on every attempt. The shell

stayed responsive and the reporter kept printing throughout the 48,450 denials above, which is the observable form of that claim.

It is fair to say the denied program still burns its own share of CPU while spinning. That is app_dev's retry loop, not the permission check, and it is the program's own slice to waste.

---

6. Shell

```
perms                    list every running application and what it may touch
perms <app> <dev> on|off  grant or revoke one device
```

The listing exists because a capability nobody can see is a capability nobody audits. Grants come from a source file, so without this, checking what a running program holds means reading `kmain.c` and trusting that the build on the board matches it. This asks the kernel.

Devices are listed **by name, not as a hex mask**. A mask is exactly the kind of thing that gets misread on the wrong day.

The mutator exists to make refusals testable — §5.2 is only possible because of it — and it revokes from a *running* program, which is the whole claim.

Note what is deliberately absent: **a program cannot grant itself anything.** There is no `sys device` operation that reaches `device_grant()`. Every grant comes from the kernel side, from this command or from the launch table.

---

7. Status

Claim	Evidence
Grants match the launch table	§5.1 — `perms=0x00000025`
Check is live, not launch-time	§5.2 — revoked mid-run, readings ceased
Refusal is not a fault	§5.2 — program survived 48,450 denials
Grants are restorable	§5.2 — resumed and completed after `perms 0 0 on`
Slots do not inherit capabilities	§5.3 — `counter` in a slot that held `sd`
Denied programs do not starve others	§5.4 — shell responsive throughout
Caller banks are separate	§5.2 — shell read `light` while `dev` was denied
A program cannot grant itself	§6 — no syscall reaches `device_grant()`

Open, and explicitly not claimed:

- **No image identity.** Until it exists this is containment, not security (§3).
- `store` **banks are keyed on a reusable slot** (§3). Same fix, same prerequisite.